Concurrent Orchestration

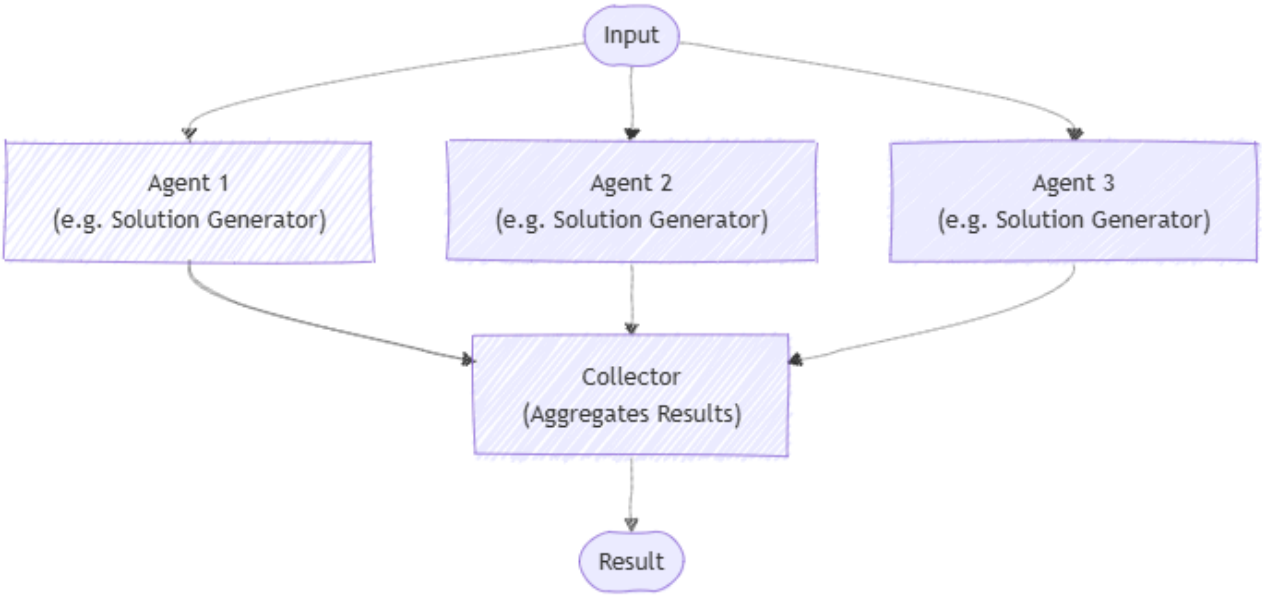
> **Important**
>
> Agent Orchestration features in the Agent Framework are in the experimental stage. They are under active development and may change significantly before advancing to the preview or release candidate stage.

Concurrent orchestration enables multiple agents to work on the same task in parallel. Each agent processes the input independently, and their results are collected and aggregated. This approach is well-suited for scenarios where diverse perspectives or solutions are valuable, such as brainstorming, ensemble reasoning, or voting systems.

To learn more about the pattern, such as when to use the pattern or when to avoid the pattern, see Concurrent orchestration.

Common Use Cases

Multiple agents generate different solutions to a problem, and their responses are collected for further analysis or selection:



What You'll Learn

- How to define multiple agents with different expertise

- How to orchestrate these agents to work concurrently on a single task
- How to collect and process the results

Define Your Agents

Agents are specialized entities that can process tasks. Here, we define two agents: a physics expert and a chemistry expert.

> **Tip**
>
> The ChatCompletionAgent is used here, but you can use any agent type.

C#

```
using Microsoft.SemanticKernel;
using Microsoft.SemanticKernel.Agents;
using Microsoft.SemanticKernel.Agents.Orchestration;
using Microsoft.SemanticKernel.Agents.Orchestration.Concurrent;
using Microsoft.SemanticKernel.Agents.Runtime.InProcess;

// Create a kernel with an AI service
Kernel kernel = ...;

ChatCompletionAgent physicist = new ChatCompletionAgent{
    Name = "PhysicsExpert",
    Instructions = "You are an expert in physics. You answer questions from a physics perspective."
    Kernel = kernel,
};

ChatCompletionAgent chemist = new ChatCompletionAgent{
    Name = "ChemistryExpert",
    Instructions = "You are an expert in chemistry. You answer questions from a chemistry perspective."
    Kernel = kernel,
};
```

Set Up the Concurrent Orchestration

The `ConcurrentOrchestration` class allows you to run multiple agents in parallel. You pass the list of agents as members.

C#

```
ConcurrentOrchestration orchestration = new (physicist, chemist);
```

Start the Runtime

A runtime is required to manage the execution of agents. Here, we use `InProcessRuntime` and start it before invoking the orchestration.

C#

```
InProcessRuntime runtime = new InProcessRuntime();
await runtime.StartAsync();
```

Invoke the Orchestration

You can now invoke the orchestration with a specific task. The orchestration will run all agents concurrently on the given task.

C#

```
var result = await orchestration.InvokeAsync("What is temperature?", runtime);
```

Collect Results

The results from all agents can be collected asynchronously. Note that the order of results is not guaranteed.

C#

```
string[] output = await result.GetValueAsync(TimeSpan.FromSeconds(20));
Console.WriteLine($"# RESULT:\n{string.Join("\n\n", output.Select(text => $"
{text}"))}");
```

Optional: Stop the Runtime

After processing is complete, stop the runtime to clean up resources.

C#

```
await runtime.RunUntilIdleAsync();
```

Sample Output

plaintext

```
# RESULT:
Temperature is a fundamental physical quantity that measures the average kinetic en-
ergy ...

Temperature is a measure of the average kinetic energy of the particles ...
```

> **Tip**
>
> The full sample code is available **here**

Next steps

Sequential Orchestration

Last updated on 07/22/2025